

Sander VanWilligen

Polk City, Iowa | sam.vanwilligen@gmail.com | (515) 451-0262 | github.com/sandervw

PROFESSIONAL SUMMARY

Full-stack engineer with 10 years shipping production systems in TypeScript, Python, and SQL. Builds the whole path: React front ends and Canvas renderers, Node/Express and FastAPI services with hand-rolled auth, session, and rate-limit layers, Postgres and MongoDB schemas underneath, and the Terraform, containers, and CI that put them on the internet. Currently leads an engineering team, owning architecture, hiring, and the security framework that cleared agentic coding tools for production.

WORK EXPERIENCE

Lead Data Engineer

Pharmacists Mutual - Algona, Iowa

March 2026 - Present

- Lead the engineering team: architectural decisions, code standards, hiring, and onboarding.
- Championed agentic coding tool adoption org-wide; ran the evaluation and authored the security framework (guardrails, approval flows, and a custom SQL MCP server) that cleared it for production.
- Set architectural direction for a platform migration from on-prem SQL Server to Azure and Microsoft Fabric.
- Recruited and onboarded a senior engineer, converting undocumented single-owner systems into documented, team-owned services.

Senior Software Engineer

Pharmacists Mutual - Algona, Iowa

2024 - March 2026

- Rebuilt the enterprise reporting platform from scratch, replacing opaque vendor logic with documented, tested internal systems; cut storage 50% and processing time over 60%.
- Redesigned ingestion to flow through a master data management layer, integrating a third-party platform into the existing pipeline.
- Sole technical decision-maker for architecture and strategy, reporting directly to the C-suite.

Software Engineer

Pharmacists Mutual - Algona, Iowa

2020 - 2024

- Took full ownership of the ETL, SQL architecture, and reporting stack, moving the team off a failed third-party vendor onto an internally built platform.
- Designed and maintained SQL Server databases, integration packages, and the models powering executive dashboards.

Developer

Pharmacists Mutual - Algona, Iowa

January 2018 - 2020

- Built a Java service integrating Paymentus for automated payments and paperless billing.
- Initiated the rebuild of the reporting architecture after the incumbent vendor ceased operations.

IT Contractor / Intern

Pharmacists Mutual - Algona, Iowa

Summer 2016 - December 2017

- Wrote a nightly Java process running government OFAC compliance screenings; cleared the full reporting backlog; supported internal and external .NET applications.

EDUCATION

B.S. Software Engineering

Iowa State University - Ames, Iowa

Graduated December 2017

Dean's List - Fall 2013, Spring 2014, Spring 2015, Fall 2015, Spring 2016

SKILLS

Languages: TypeScript, JavaScript, Python, SQL/T-SQL, Java, C# (.NET), HTML/CSS, PowerShell

Frontend: React 19, React Router, Context and custom hooks, Vite, Astro, Canvas 2D, @dnd-kit, TipTap, responsive and accessible UI

Backend & APIs: Node.js, Express 5, FastAPI, Cloudflare Workers/Pages Functions, REST contract design, Mongoose, asyncpg, Zod, Pydantic, JWT, bcrypt/argon2id, HMAC, session and cookie security, rate limiting, Helmet

Testing: Vitest, React Testing Library, MSW, Supertest, pytest, MongoDB Memory Server, coverage reporting

Data: MongoDB, Postgres, SQL Server, DuckDB, dbt Core, Dagster, polars, pandas, spaCy

Cloud & Infrastructure: Docker/Compose, Terraform/OpenTofu, Cloudflare (Pages, Workers, Tunnel, Access, R2), Azure, GitHub Actions CI/CD, systemd, Linux

AI Engineering: Claude API/SDK, OpenAI API, Model Context Protocol servers, agentic development workflows

PROJECTS

Leaves (github.com/sandervw/LeavesApp)

Full-stack writing tool built on a tree data model: users compose reusable template trees, then drag one onto the workspace to instantiate an entire story tree. Node/TypeScript/Express 5 REST API in a layered routes-controllers-services-models architecture, with a generic `TreeService<T>` base class that `Template` and `Storynode` services extend for recursive traversal, depth-capped descendant deletes, and word-weight rollups. Mongoose discriminators keep both node kinds in one collection; Zod validates every request; JWT access and rotated refresh tokens ride `httpOnly` cookies over `bcrypt` hashing, behind `Helmet`, `CORS`, and `express-rate-limit`. React 19/Vite/React Router 7 front end with five Context providers, custom hooks over an `Axios` client, `@dnd-kit` drag-and-drop, and a `TipTap` markdown editor. Tested with Vitest across unit, schema, and Supertest integration suites against an in-memory MongoDB, plus React Testing Library and MSW. Deployed to Azure Static Web Apps and Container Apps with secrets in Key Vault via managed identity.

Dying Skies (github.com/sandervw/dying-skies, dying skies.com)

Procedural universe, four services under one contract. React 19/TypeScript/Vite front end driving a Canvas 2D render loop: `requestAnimationFrame` stepping with clamped deltas, a `devicePixelRatio`-aware backing store, coalesced resizes, hit-testing on rotated sprites, and seeded generative music via `Tone.js`.

FastAPI/asyncpg/Postgres backend issues 256-bit HMAC-derived star seeds with constant-time tag verification, `argon2id` auth behind `httpOnly` session cookies capped at four devices, and rate limiting that trusts `X-Forwarded-For` only to a configured proxy depth; covered by `pytest` and containerized with Docker Compose. OpenTofu provisions the VPS, `systemd` units, and Cloudflare Tunnel/Access; GitHub Actions deploys only the services a push touches.

Data-Team-Tarot (github.com/sandervw/Data-Team-Tarot, cheddarsoap.com)

Astro 6 static site with a React 19 island, served on Cloudflare Pages. Two TypeScript Cloudflare Workers: a site-wide auth gate that signs and verifies its own session cookie with Web Crypto HMAC-SHA256, and a submissions API that validates payloads, commits new entries through the GitHub Contents API with retry on 409/422 conflicts, and triggers the next build.

Gutenberg-Fingerprint (github.com/sandervw/Gutenberg-Fingerprint, gufime.com)

Nightly change-data-capture pipeline in plain Python, operated solo on one ~\$5/month VPS. Watermark-ledger diffing against the Project Gutenberg catalog, a rate-limited fetch layer with mirror fallback and terminal-failure states, spaCy parsing that measures 63 stylometric series per book, and a storage seam that runs the same code against DuckDB locally and Postgres in production. Orchestrated as a Dagster asset graph with a sensor that skips the chain on quiet nights; dbt models and an Evidence.dev SPA published to Cloudflare, all provisioned with OpenTofu.